

Debug Information Validation for Optimized Code

PLDI '20 – Yuanbo Li, Shuo Ding, Qirun Zhang, Davide Italiano

Silas Kraume

Heinrich-Heine-Universität Düsseldorf

08.07.2026

Agenda

- 1 Motivation
- 2 Hintergrund: Debug-Informationen
- 3 Problemstellung
- 4 Formale Definitionen
- 5 Konkrete Bug-Beispiele
- 6 Methodik: Shadow Validation & Line Augmentation
- 7 Testalgorithmus
- 8 Evaluation & Ergebnisse
- 9 Fallstudien
- 10 Fazit & Zukünftige Arbeit

Warum optimierten Code debuggen?

1 Motivation

- ▶ Fast alle Produktionssoftware wird mit Optimierungen kompiliert
- ▶ Industriebeispiele:
 - ▷ GNU Autotools: Standard `-g -O2` in Release-Builds
 - ▷ Red Hat Developer Guide: Empfiehlt `-g -O2` für alle Entwickler
 - ▷ ...
- ▶ Post-Mortem Debugging: Analyse von Coredumps nach Crashes in Produktion
- ▶ Performance-Bugs: Treten nur in optimiertem Code auf
- ▶ Historischer Kontext: Vor 20 Jahren: "Optimierungen ausschalten zum Debuggen"
Heute: Debuggen von optimiertem Code ist obligatorisch

Was fehlt im aktuellen Testing?

1 Motivation

- ▶ Bisheriger Testing-Fokus:
 - ✓ Compiler-Korrektheit (Maschinencode-Ausgabe)
 - ✓ Debugger-Funktionalität (interaktives Testing)
 - ✗ Korrektheit der Debug-Informationen für optimierten Code
- ▶ Das Problem: Compiler können generieren:
 - ✓ Korrekten Maschinencode
 - ✗ Falsche Debug-Informationen
- ▶ Konsequenz: Debugger zeigt falsche Werte, obwohl Programm korrekt läuft
- ▶ Forschungsfrage: Generieren Compiler immer korrekte Debug-Informationen für optimierten Code?

Was sind Debug-Informationen?

2 Hintergrund: Debug-Informationen

- ▶ Zweck: Stellt Verbindung zwischen Quellcode und Maschinencode her
- ▶ Generiert von: Compilern mit -g Flag
- ▶ Formate:
 - ▷ DWARF (Linux/Unix – am häufigsten, Fokus dieses Papers)
 - ▷ PDB (Windows)
 - ▷ STABS (Ältere Unix-Systeme)
- ▶ Verwendet von: Debuggern (GDB, LLDB) zur Interpretation laufender Programme
- ▶ Enthält: Funktionen, Variablen, Zeilennummern, Typen, Speicherorte
- ▶ Kritisch: Debugger verlassen sich vollständig auf diese Informationen zur Anzeige von Variablenwerten

DWARF Debug-Information Beispiel

2 Hintergrund: Debug-Informationen

```
1 DW_TAG_compile_unit // Kompilierungseinheit-Metadaten ('Wurzelknoten')
2   DW_AT_producer ("clang ver 7.0.1")
3   DW_AT_language (DW_LANG_C99)
4   DW_AT_name ("a.c")
5   DW_AT_low_pc (0x00400480) // Startadresse der Funktion
6   DW_AT_high_pc (<offset-from-lowpc>50) // Endadresse der Funktion
7
8 DW_TAG_subprogram // Funktionsinformationen
9   DW_AT_name ("main")
10  DW_AT_decl_line (2)
11  DW_AT_frame_base (DW_OP_reg6 RBP)
12
13 DW_TAG_variable // Variableninformationen
14   DW_AT_name ("a")
15   DW_AT_location (DW_OP_fbreg -8) // Offset von RBP (Variablenort)
16   DW_AT_decl_line (1) // Deklarationszeile im Quellcode
17   DW_AT_type (0x00000060 "int") // Typ-Referenz (z.B. int)
18
19 DW_TAG_variable // Variableninformationen
20   DW_AT_name ("b")
21   DW_AT_location (DW_OP_fbreg -12)
22   DW_AT_decl_line (1)
23   DW_AT_type (0x00000060 "int")
```

Problemstellung – Kernherausforderung

3 Problemstellung

- ▶ Kernproblem:
 - ▷ Generieren von geeigneten Eingabeprogrammen P für systematisches Debug-Info-Testing
- ▶ Warum existierende Techniken versagen:
 - ▷ Compiler-Testing generiert Programme mit festen Ausgabe-Statements (print, return)
 - ▷ Programme werden nur einmal ausgeführt bzgl. Programmeingabe
 - ▷ Debugging ist interaktiver – Benutzer inspizieren mehrere Programmpunkte mit vielen Variablen

Herausforderungen bei Debug-Info-Validierung

3 Problemstellung

► 1. Code Location Problem:

- ▷ Compiler-Optimierung rearrangiert Instruktionen und eliminiert Statements
- ▷ Viele Programmpunkte im Quellcode könnten wegoptimiert werden
- ▷ Breakpoints an diesen Orten stoppen Programmausführung nicht notwendigerweise

► 2. Data Value Problem:

- ▷ Debugger können jede Variable an einem spezifischen Programmpunkt inspizieren
- ▷ Ausgabe einer beliebigen Variable kann undefined behaviors (UB) verursachen
- ▷ Beispiele: Zugriff auf uninitialisierte Variable, Out-of-Bound-Array-Element

Definition – Actionable Program

4 Formale Definitionen

Definition (Actionable Program)

Für einen spezifischen Optimierungsprozess enthält ein **actionable program** $P_{\langle s, v \rangle}$ einen Programmort s und eine Variable v , sodass sowohl im unoptimierten Programm als auch im optimierten Programm $P'_{\langle s, v \rangle}$ der Debugger an s stoppen kann und die Ausgabe von v an s wohldefiniert ist (kein undefined behaviour).

- Menge der Actionable Programs:

$$|\mathcal{P}| = \sum_{i \in I} s_i \cdot v_i$$

Wobei:

- ▷ I = Menge der ausgeführten Zeilen
- ▷ s_i = Programmort
- ▷ v_i = Anzahl der wohldefinierten Variablen an jedem Ort

Definition – Korrektheit

4 Formale Definitionen

- ▶ Value-Printing Sequence: $A_{\langle s, v \rangle}$ repräsentiert $\text{break}(s) \rightarrow \text{run} \rightarrow \text{print}(v) \rightarrow \text{quit}$
- ▶ Debugger-Ausgabe: $\text{DBG}(P, A_{\langle s, v \rangle})$ gibt den Wert von v nach Abschluss der Debugging-Session aus
- ▶ Referenzwert: $\text{PRINT}(P, s, v)$ bezeichnet den ausgegebenen Referenzwert
 - ▷ Print-Statement einfügen, um Wert von v an s im Quellcode anzuzeigen
 - ▷ Programm P ohne Optimierung kompilieren und ausführen

Definition (Korrektheit)

Die Debug-Informationen für optimierten Code P' sind korrekt gdw.

$$\text{DBG}(P', A_{\langle s, v \rangle}) \equiv \text{PRINT}(P, s, v)$$

wobei P ein unoptimiertes Programm und P' ein optimiertes Programm aus demselben Quellcode ist.

Definition – Korrektheit anwenden – Konkrete Szenarien

5 Konkrete Bug-Beispiele

Szenario	$PRINT(P, s, v)$	$DBG(P', A_{\langle s, v \rangle})$	Urteil
Korrekte Debug-Info	5	5	✓ Korrekt
Falsche Debug-Info	5	1	✗ BUG ($5 \neq 1$)
Variable Optimized Out	5	null	✓ Akzeptabel
Debugger Crash	5	crash	✗ BUG (Debugger sollte nicht crashen)

- ▶ Wichtig: “Optimized out” ist kein Bug
- ▶ Bug-Kriterium: Nur falsche Werte (nicht null) verletzen Korrektheit
- ▶ Paper-Fokus: Testing der Korrektheit, nicht Vollständigkeit

LLVM Bug 43893 (Clang-9)

5 Konkrete Bug-Beispiele

```
3 int c = --a;  
4 int l = -8L;  
5 l = c;          // optimized out  
6 c > (b = 0); // l
```

Erwartet: Variable l sollte -1 in Zeile 6 sein

Tatsächlich (LLDB, -g -O2): Zeigt -8

DWARF Bug: InstCombine-Pass
verwirft ungenutzte Variable,
markiert Debug-Info aber nicht als
unavailable

Korrektheitsverletzung:

$PRINT(P, \text{Zeile } 6, l) = -1$

$DBG(P', A_{\langle 6, l \rangle}) = -8$

$-1 \neq -8 \rightarrow \text{BUG (Definition verletzt)}$

Status: In Development-Version gefixt

GCC Bug 89463 (GCC-8.3)

5 Konkrete Bug-Beispiele

```
3  int i;  
4  for (; a < 10; a++)  
5      i = 0;  
6  for (; i < 6; i++)  
7      ;  
8  opt_me_not(); // i
```

Erwartet: Variable i sollte 6 in Zeile 8 sein
Tatsächlich (GDB, -g -O3): Zeigt 0 (falsch!)

Ursache: Dead Code Elimination
bereinigt
degenerierte Phi-Nodes nicht

Korrektheitsverletzung:
 $PRINT(P, \text{Zeile } 8, i) = 6$
 $DBG(P', A_{\langle 8, i \rangle}) = 0$
 $6 \neq 0 \rightarrow \text{BUG (Definition verletzt)}$

Status: In Development-Version gefixt

Actionable Program Transformation

6 Methodik: Shadow Validation & Line Augmentation

► Originalprogramm P :

```
1 char a,b;  
2 int main() {  
3     unsigned c;  
4     --b;  
5     c=2;  
6     if (a)  
7         b=0;  
8     return 0;  
9 }
```

- Actionable Program $P_{\langle 6,c \rangle}$: Breakpoint in Zeile 6, inspiziere Variable c ✓
- Actionable Program $P_{\langle 5,b \rangle}$: Füge `opt_me_not()` in Zeile 5 ein, inspiziere b ✓
- Non-Actionable Program $P_{\langle 4,c \rangle}$: Variable c uninitialisiert in Zeile 4 ✗

Systematisches Testing Framework Übersicht

6 Methodik: Shadow Validation & Line Augmentation

Eingabe: Wohlgeformtes Seed-Programm P_w

- ▶ Schritt 1: Pre-Processing
 - ▷ Sammle Line Coverage Information S
 - ▷ Baue Variablenmengen V_s für jede Zeile (AST-Traversierung)
- ▶ Schritt 2: Actionable Program Generierung
 - ▷ Shadow Validation (löse Data Value Problem)
 - ▷ Line Augmentation (löse Code Location Problem)
 - ▷ Generiere Menge der actionable programs $\mathcal{P}$
- ▶ Schritt 3: Systematisches Testing
 - ▷ Kompiliere unoptimiert → Erhalte Referenz $PRINT(P, s, v)$
 - ▷ Kompiliere optimiert → Erhalte Debugger-Ausgabe $DBG(P', A_{\langle s, v \rangle})$
 - ▷ Vergleiche Ausgabe → Reporte Bug wenn $DBG \neq PRINT$ und $DBG \neq \text{null}$

Shadow Validation Technik (Data Value Problem)

6 Methodik: Shadow Validation & Line Augmentation

- ▶ Ziel: Sicherstellen, dass Inspizieren von Variable v an Zeile s sicher ist (kein UB)
- ▶ Methode:
 - ▷ Erstelle "Shadow Program" P_1 durch Einfügen von `print(v)` an Zeile s
 - ▷ Führe P_1 mit dynamischer Analyse aus
 - ▷ Erkenne undefined behaviors (uninitialisierte Reads, Out-of-Bounds, etc.)
- ▶ Ergebnis:
 - ▷ Wenn UB erkannt → Kandidat verwerfen (non-actionable)
 - ▷ Wenn sicher → Fahre fort mit Line Augmentation
- ▶ Tools: CompCert Interpreter + Clang UndefinedBehaviorSanitizer

Shadow Validation Beispiel

6 Methodik: Shadow Validation & Line Augmentation

```
3 unsigned c;  
4 --b;  
5 c=2; // c hier initialisiert
```

► Shadow Program P_1 (Invalide):

```
3 unsigned c;  
4 --b;  
5 printf("%d", c); // UB! c hier uninitialisiert  
6 c=2; // c hier initialisiert
```

▷ Abgelehnt durch Shadow Validation (non-actionable)

► Shadow Program P_1 (Valide):

```
3 unsigned c;  
4 --b;  
5 c=2; // c hier initialisiert  
6 printf("%d", c); // sicher, c hier initialisiert
```

▷ Akzeptiert durch Shadow Validation (actionable)

Line Augmentation Technik (Code Location Problem)

6 Methodik: Shadow Validation & Line Augmentation

- ▶ Ziel: Sicherstellen, dass Debugger an Zeile `s` stoppen kann, auch wenn optimiert
- ▶ Methode:
 - ▷ Prüfe DWARF Line Number Table für Zeile `s`
 - ▷ Wenn Zeile `s` fehlt/optimiert:
 - Füge Aufruf von unoptimierbarer Funktion `opt_me_not()` vor Zeile `s` ein
 - Dies wirkt als Barriere für Optimierung
 - Füge Meta-Information (Kommentare) hinzu, um Variable `v` zu spezifizieren
- ▶ Implementierung: `opt_me_not()` in separater Kompilierungseinheit (verhindert Inlining)
- ▶ Alternative: `__asm__ volatile("" ::: "memory")` als zusätzliche Barriere

Systematischer Testalgorithmus

7 Testalgorithmus

Systematischer Testalgorithmus

Eingabe: Ein wohlgeformtes Testprogramm P_w

1. $\mathcal{P} \leftarrow \text{ActionableProgramGeneration}(P_w)$
2. **for each** $P_{\langle s, v \rangle} \in \mathcal{P}$ **do**
 - 2.1 $A_{\langle s, v \rangle} \leftarrow \text{ExtractActions}(P_{\langle s, v \rangle})$
 - 2.2 $P' \leftarrow$ kompiliere $P_{\langle s, v \rangle}$ mit Optimierung und “-g”
 - 2.3 **if** $\text{DBG}(P', A_{\langle s, v \rangle})$ ist nicht null **and**
 $\text{DBG}(P', A_{\langle s, v \rangle}) \neq \text{PRINT}(P, s, v)$ **then**
 - 2.3.1 Reporte Bug mit Programm $P_{\langle s, v \rangle}$,
Action $A_{\langle s, v \rangle}$ und verwendetem Optimierungs-Flag

Bug-Zusammenfassung

8 Evaluation & Ergebnisse

Compiler	Reportet	Gefixt	Duplikate	Invalid
GCC	21	4	3	2
LLVM/Clang	26	7	0	1
Gesamt	47	11	3	3

- ▶ Bestätigte Bugs Gesamt: 47 (GCC + LLVM)
- ▶ GCC: 21 Bugs reportet
4 Gefixt, 3 Duplikate, 2 Invalid, 21 Wrong Output, 0 Crashes
- ▶ LLVM/Clang: 26 Bugs reportet
7 Gefixt, 0 Duplikate, 1 Invalid, 23 Wrong Output, 3 Crashes
- ▶ Status: 11 Bugs zum Zeitpunkt der Veröffentlichung gefixt (23 %)
- ▶ Bestätigungsrate: $47/47 = 100\%$ (alle Bugs von Entwicklern bestätigt)

Betroffene Compiler-Versionen

8 Evaluation & Ergebnisse

- ▶ Betroffene GCC-Versionen:
 - ▷ GCC 6.X, 7.X, 8.X, 9.X, Trunk
 - ▷ Einige Bugs datieren zurück zu GCC 4.4 (9 Jahre alt!)
- ▶ Betroffene LLVM-Versionen:
 - ▷ LLVM 5.X, 6.X, 7.X, 8.X, 9.X, Trunk
- ▶ Implikation: Dies ist ein anhaltendes Problem, nicht historisch
- ▶ Latente Bugs: Einige Probleme existierten Jahre vor Entdeckung

Betroffene Optimierungs-Level

8 Evaluation & Ergebnisse

- ▶ GCC: Bugs bei -O1, -O2, -O3, -Og
 - ▷ Sogar -Og (optimiert für Debugging) hat Bugs!
- ▶ LLVM: Bugs bei -O1, -O2, -O3
 - ▷ -Og ist Wrapper für -O1 in LLVM
- ▶ Wichtige Erkenntnis: Bugs existieren auf ALLEN Optimierungs-Leveln
- ▶ Ursache: Debug-Werte falsch von Passes aktualisiert, folgende Passes löschen falsche Info nicht
- ▶ Implikation: Kein Optimierungs-Level für korrekte Debug-Info vertrauenswürdig

LLVM-Komponenten mit Bugs

8 Evaluation & Ergebnisse

Komponente	Bug-Anzahl
DWARF Parser	6
SelectionDAG	5
SROA	2
InstCombine	2
EarlyCSE	2
MachineScheduler	2

Komponente	Bug-Anzahl
LoopDeletion	1
SimplifyCFG	1
LSR	1
DeadStoreElimination	1
Inliner	1
CodegenPrepare	1

- ▶ Wichtige Erkenntnis: DWARF Parser hat meiste Bugs (inklusive 3 LLDB Crashes)
- ▶ Verteilung: Bugs über viele Optimierungs-Passes verteilt

Fallstudie: GCC wrong value bug 92387

9 Fallstudien

```
1 static int a[6];
2 int b;
3 static void c() { b = 5 ^ a[b & 5]; }
4 int main() {
5     int i, d = 0;
6     c();
7     i = 0;
8     for (; i < 8; i++) {
9         c();
10        if(d) printf("%d",i); // i
11    }
12 }
```

- ▶ Optimierung: -O1
- ▶ Korrektheitsverletzung: $PRINT(P, \text{Zeile } 10, i) = 0 \neq 1 = DBG(P', A_{\langle 10, i \rangle})$
BUG (Definition verletzt)
- ▶ Status: Gefixt

Fallstudie: LLVM Wrong Value Bug 43957

9 Fallstudien

```
1 char a, b;  
2 int main() {  
3     --b;  
4     unsigned l_801 = 36901;  
5     ++l_801;  
6     if (a) // l_801  
7         b = 0;  
8 }
```

► Optimierung: -O3

► Korrektheitsverletzung:

$PRINT(P, \text{Zeile 6}, l_801) = 36902 \neq 36901 = DBG(P', A_{\langle 6, l_801 \rangle})$

BUG (Definition verletzt)

► Status: Gefixt

Framework-Generalisierbarkeit – Rust Compiler

10 Fazit & Zukünftige Arbeit

```
1 fn main() {  
2     let mut counter = 0;  
3     {  
4         let mut tick = || counter += 1;  
5         tick();  
6         tick();  
7         lib::opt_me_not(); // counter sollte 2 sein  
8     }  
9     assert_eq!(counter, 2);  
10 }
```

- ▶ 2 bestätigte Bugs in 3 Tagen gefunden
- ▶ Bug: Rust Debugger zeigt counter = 0, sollte 2 sein
- ▶ Optimierung: -C opt-level=1
- ▶ Anpassungen nötig:
 - ▷ Grammar-Instrumentierung (Rust vs. C)
 - ▷ Kompilierungseinheit = crate (nicht Datei)
 - ▷ Rust's safe Code hilft bei UB-Erkennung

- ▶ Problem: Debug-Informationen für optimierten Code sind oft inkorrekt
- ▶ Lösung: Actionable Program Framework mit formalen Definitionen
 - ▷ Algorithmus: Systematisches Testing
- ▶ Auswirkung: 47 Bugs in GCC/LLVM, 2 in Rust
- ▶ Community-Reaktion: 11 Bugs gefixt, 100 % Bestätigungsrate
- ▶ Zukünftige Arbeit:
 - ▷ Debugger-Korrektheit direkt testen
 - ▷ Auf mehr Sprachen erweitern
 - ▷ Vollständigkeit testen (fehlende Debug-Info vs. falsche Info)

Abschließende Gedanken

10 Fazit & Zukünftige Arbeit

LLVM-Entwickler (Bug 39896)

“Thanks for the PR, this was likely a serious quality-of-debug-experience issue for anyone hit by it.”

DWARF-Komiteemitglied

“[Your bug reports] seem clear and reasonably complete. Generating correct debug info for optimized code is important.”

`cc.gatech.edu/~qrzhang/projects/debug/`

Vielen Dank für Eure Aufmerksamkeit!